

SOC ANALYST TRAINING MANUAL

ASSESSMENT 1

RDP Brute Force → Privilege Escalation → Persistence → C2 → Lateral Movement

EXTRACT: PART 1 — THEORY & PART 2 — WINDOWS EVENT IDs

SANS/GIAC-Style SOC Blue-Team Lab Workbook

CONFIDENTIAL — FOR TRAINING PURPOSES ONLY

PART 1 — THEORY

1.1 Scenario Overview

Field	Details
Company	FinSecure Corp — Mid-size financial services firm (~500 employees)
Industry	Finance / Banking
Headquarters	New York, NY
Environment	Hybrid (on-prem AD + Azure AD)
Incident Type	Intrusion — RDP Brute Force → Lateral Movement
Severity	CRITICAL
Analyst Role	L2 SOC Analyst

Network Architecture

Zone	Subnet	Key Systems
DMZ	10.0.0.0/24	Firewall, WAF, Reverse Proxy
Corporate LAN	192.168.1.0/24	Workstations, Printers
Server LAN	192.168.10.0/24	DC01, FILE01, SQL01
Management	10.10.10.0/28	SIEM, EDR Console, Backup
Attacker	185.220.101.47	External — TOR Exit Node

Systems Involved

- DC01 (192.168.10.5) — Windows Server 2019 Domain Controller
- WRK-JSMITH (192.168.1.22) — Windows 10 workstation, user: jsmith
- FILE01 (192.168.10.15) — Windows Server 2016 File Server
- SIEM — Microsoft Sentinel + Elastic SIEM
- EDR — CrowdStrike Falcon

1.2 Attack Overview — RDP Brute Force

Remote Desktop Protocol (RDP) brute force is one of the most common initial access techniques in enterprise intrusions. Attackers scan the internet for open port 3389, then systematically attempt credentials using password lists or credential stuffing from previously breached databases.

Why Attackers Use RDP Brute Force

- RDP port 3389 is frequently exposed to the internet
- Many organizations use weak or default passwords
- Successful RDP login provides full interactive desktop access
- RDP sessions blend in with legitimate admin activity
- Low cost — automated tools do the work

Common Attacker Tools

Tool	Type	Notes
Hydra	Brute Force	Multi-protocol; very common in CTF & real attacks
Ncrack	Brute Force	Nmap project; RDP/SSH/FTP
Crowbar	Brute Force	Specifically designed for RDP
Metasploit	Framework	auxiliary/scanner/rdp/rdp_login
xFreeRDP	Client	Used to validate credentials & connect
BloodHound	Recon	Post-access AD enumeration

Real-World Examples

- 2021 Colonial Pipeline — RDP credential stuffing led to ransomware deployment
- 2020 Florida Water Treatment — RDP with no MFA was the entry point
- SamSam Ransomware — Exclusively used RDP brute force for initial access
- Dharma/Phobos Ransomware — RDP brute force as primary vector

PART 2 — WINDOWS EVENT IDS

Every Windows event below is presented in the standard SOC analysis format. Memorize the Event ID, know the normal vs malicious signature, and practice writing detection queries before your practical.

Event ID 4625 — An Account Failed to Log On

Definition: Generated whenever a logon attempt fails on the local machine. Primary event for detecting brute-force.

Purpose: Tracks failed authentication events — critical for detecting brute force, password spray, and credential stuffing attacks.

When Generated: Every failed logon attempt: wrong password, locked account, disabled account, unknown username, expired account.

Normal vs Attack Log Example

NORMAL: EventID: 4625 | Account: jsmith | Source: 192.168.1.10 LogonType: 10 (RemoteInteractive) | FailureReason: Wrong Password Count: 1-2 per day | Workstation: WRK-JSMITH

ATTACK / SUSPICIOUS: EventID: 4625 | Account: administrator | Source: 185.220.101.47 LogonType: 10 (RemoteInteractive) | FailureReason: Wrong Password Count: 847 in 4 minutes | Workstation: DC01 [BRUTE FORCE]

Key XML Fields

Field	Description
SubjectUserName	Account that attempted the logon
TargetUserName	Account that was targeted
LogonType	2=Interactive, 3=Network, 10=RemoteInteractive (RDP)
IpAddress	Source IP of the logon attempt
Status / SubStatus	Failure reason codes (0xC000006A = wrong pw)
WorkstationName	Machine where attempt originated
ProcessName	Process that initiated the logon

Important Fields to Focus On

- IpAddress — external IP = immediate escalation
- TargetUserName — admin/service accounts = high priority
- LogonType 10 = RDP (most dangerous for brute force)
- Count per minute — >20/min from same IP = brute force threshold
- Status 0xC0000064 = unknown username | 0xC000006A = wrong password

Detection Logic: Alert when: >10 failed logons from same source IP within 2 minutes, OR >5 different usernames attempted from same IP within 5 minutes (password spray), OR any failed logon from external IP on LogonType 10.

Investigation Tips

- Check if same IP succeeds after failures — confirms successful brute force
- Look for LogonType 10 from non-corporate IP ranges
- Correlate with 4624 within 60s to confirm compromise
- Check if targeted account exists in AD
- Review ASN of source IP — TOR/VPN/hosting = high risk

Common False Positives

- User mistyping password on Monday morning (1-3 failures)

- Service account with expired/changed password
- VPN reconnect with stale credentials
- Locked workstation auto-logon failure

MITRE ATT&CK Mapping: T1110.001 — Brute Force: Password Guessing | Tactic: Credential Access / Initial Access

Sigma Rule

```
title: RDP Brute Force Attack logsource: product: windows service: security detection: selection: EventID:
4625 LogonType: 10 timeframe: 2m condition: selection | count() by IpAddress > 10 level: high tags: -
attack.credential_access - attack.t1110.001
```

Splunk Query

```
index=wineventlog EventCode=4625 LogonType=10 | stats count by src_ip, user, dest | where count > 10 |
sort -count
```

Microsoft Sentinel KQL

```
SecurityEvent | where EventID == 4625 and LogonType == 10 | summarize FailCount=count() by IpAddress,
TargetUserName, Computer | where FailCount > 10 | order by FailCount desc
```

■ **SOC Note:** When you see 4625 spike, IMMEDIATELY check if a 4624 follows from the same IP. Brute force without a follow-up 4624 = failed attempt. 4625 then 4624 = confirmed compromise. Escalate to P1 immediately.

Interview Questions

1. What LogonType value indicates an RDP logon attempt?
2. What is the difference between Status 0xC0000064 and 0xC000006A?
3. At what threshold would you escalate a 4625 pattern to P1?
4. How do you differentiate a brute force from a user forgetting their password?

Event ID 4624 — An Account Was Successfully Logged On

Definition: Generated every time a logon succeeds. After a brute force sequence of 4625s, a 4624 indicates the attacker has gained access.

Purpose: Confirms authentication success. Critical for detecting post-brute-force access, lateral movement, pass-the-hash, and unauthorized access.

When Generated: Every successful local or remote logon to the system.

Normal vs Attack Log Example

NORMAL: EventID: 4624 | Account: jsmith | Source: 192.168.1.10 LogonType: 10 | AuthPackage: NTLM | WorkHours: 09:15 Workstation: WRK-JSMITH | ProcessName: winlogon.exe

ATTACK / SUSPICIOUS: EventID: 4624 | Account: administrator | Source: 185.220.101.47 LogonType: 10 | AuthPackage: NTLM | Time: 03:42 AM Workstation: DC01 | After 847 failures [BRUTE FORCE SUCCESS]

Key XML Fields

Field	Description
TargetUserName	Account that successfully logged on
LogonType	2=Console, 3=Network, 10=RDP, 7=Unlock
IpAddress	Source IP of successful logon
AuthenticationPackageName	NTLM vs Kerberos — NTLM is suspicious for RDP
LogonProcessName	NtLmSsp or Kerberos
ElevatedToken	%%1842 = Yes = admin token issued

Important Fields to Focus On

- Logon after multiple 4625 = confirmed brute force success
- LogonType 10 from external IP = critical finding
- ElevatedToken %%1842 = admin access granted
- AuthPackage NTLM for domain admin = suspicious (should be Kerberos)
- Time of day — 2-4 AM from unusual IP = IOC

Detection Logic: Alert when: 4624 LogonType=10 from external/untrusted IP, OR 4624 within 5 minutes of >10x 4625 from same source, OR 4624 for privileged account from new IP.

Investigation Tips

- Always check 30 minutes before the 4624 for 4625 pattern
- Verify if IpAddress is in approved remote access list
- Check ElevatedToken field — admin token = higher risk
- Look for 4688 process creation immediately after — shows what attacker did
- Check WorkstationName — should match known asset inventory

Common False Positives

- Legitimate admin RDP from home/VPN (check VPN logs)
- Service account network logon Type 3 (very common)
- Scheduled task execution (LogonType 5)
- Batch job logon (LogonType 4)

MITRE ATT&CK Mapping: T1078 — Valid Accounts | Tactic: Initial Access / Lateral Movement / Persistence

Sigma Rule

```
title: Successful RDP From External IP detection: selection: EventID: 4624 LogonType: 10 filter: IpAddress|startswith: - '192.168.' - '10.' condition: selection and not filter level: high
```

Splunk Query

```
index=wineventlog EventCode=4624 LogonType=10 | where NOT match(src_ip, "^192\.168\.|^10\.") | table  
_time, user, src_ip, dest, LogonType, AuthPackage
```

Microsoft Sentinel KQL

```
SecurityEvent | where EventID == 4624 and LogonType == 10 | where IPAddress !startswith "192.168." and  
IPAddress !startswith "10." | project TimeGenerated, TargetUserName, IPAddress, Computer, LogonType
```

■ SOC Note: Always correlate 4624 with prior 4625 events. A lone 4624 from external IP with no prior failures could indicate stolen credentials (credential stuffing). Both are critical — brute force and pre-stolen creds need different responses.

Interview Questions

1. What LogonType values are associated with lateral movement?
2. Why is NTLM authentication for a domain admin account suspicious?
3. What does ElevatedToken %%1842 indicate?
4. How quickly after 4624 should you look for follow-up events?

Event ID 4672 — Special Privileges Assigned to New Logon

Definition: Generated when a logon session is assigned one or more sensitive privileges such as SeDebugPrivilege, SeTcbPrivilege, SeImpersonatePrivilege.

Purpose: Indicates a privileged logon session. When seen after a 4624 from external IP, it confirms attacker has admin-level access.

When Generated: Immediately after a privileged account successfully logs on (4624). Will not appear for standard user logons.

Normal vs Attack Log Example

NORMAL: EventID: 4672 | Account: DOMAIN\sysadmin | LogonID: 0x3E4A2 Privileges: SeBackupPrivilege, SeRestorePrivilege Time: 09:22 AM | Source: 192.168.10.100 (jump server)

ATTACK / SUSPICIOUS: EventID: 4672 | Account: DOMAIN\administrator | LogonID: 0x7F2C1 Privileges: SeTcbPrivilege, SeDebugPrivilege, SeImpersonatePrivilege Time: 03:42 AM | Source: 185.220.101.47
[PRIVILEGED EXTERNAL LOGON]

Key XML Fields

Field	Description
SubjectUserName	Account receiving the privileges
SubjectLogonId	Links this event to the 4624 LogonID
PrivilegeList	List of sensitive privileges assigned

Important Fields to Focus On

- PrivilegeList containing SeDebugPrivilege = can inject into any process
- SeTcbPrivilege = Act as part of OS — extremely dangerous
- SeImpersonatePrivilege = impersonate any user token
- SubjectLogonId — link back to 4624 to confirm source IP

Detection Logic: Alert when 4672 follows a 4624 from external IP, OR when SeDebugPrivilege/SeTcbPrivilege appear for non-admin service accounts.

Investigation Tips

- Use SubjectLogonId to link to the 4624 that created this session
- High-sensitivity privileges (SeDebug, SeTcb) = immediate escalation
- Check if account is supposed to have these privileges
- Verify with AD — is this a domain admin or just local admin?

Common False Positives

- Legitimate admin logon during change window
- Antivirus/EDR service accounts (require SeDebugPrivilege normally)
- Scheduled maintenance by IT team

MITRE ATT&CK Mapping: T1078.002 — Valid Accounts: Domain Accounts | Tactic: Privilege Escalation / Persistence

Sigma Rule

```
title: Privileged Logon with SeDebugPrivilege detection: selection: EventID: 4672 PrivilegeList|contains: 'SeDebugPrivilege' condition: selection level: medium
```

Splunk Query

```
index=wineventlog EventCode=4672 | search PrivilegeList=*SeDebugPrivilege* OR PrivilegeList=*SeTcbPrivilege* | table _time, user, PrivilegeList, src_ip
```

Microsoft Sentinel KQL

```
SecurityEvent | where EventID == 4672 | where PrivilegeList has "SeDebugPrivilege" or PrivilegeList has "SeTcbPrivilege" | project TimeGenerated, SubjectUserName, PrivilegeList, Computer
```

■ SOC Note: 4625 x N → 4624 → 4672 is the classic brute-force success chain. Document the LogonID from 4672 — it will appear in every subsequent event from that privileged session, making timeline reconstruction much easier.

Interview Questions

1. What privilege allows a process to read memory of any running process?
2. How do you link a 4672 event back to the originating 4624?
3. Why does `SeImpersonatePrivilege` matter to an attacker?

Event ID 4688 — A New Process Has Been Created

Definition: Generated every time a new process is created. With command-line auditing enabled, this includes full command-line arguments.

Purpose: Reveals every command the attacker executed after gaining access. Process creation chain (parent→child) shows attacker's kill chain.

When Generated: Every process creation. Requires 'Audit Process Creation' enabled and 'Include command line in process creation events' GPO setting.

Normal vs Attack Log Example

```
NORMAL: EventID: 4688 | NewProcessName: C:\Windows\System32\cmd.exe ParentProcess: explorer.exe | User: jsmith CommandLine: cmd.exe /c dir C:\Users\jsmith\Documents
```

```
ATTACK / SUSPICIOUS: EventID: 4688 | NewProcessName: C:\Windows\System32\cmd.exe ParentProcess: svchost.exe | User: SYSTEM CommandLine: cmd.exe /c net user backdoor P@ss123! /add [ACCOUNT CREATION]
```

Key XML Fields

Field	Description
NewProcessName	Full path of the new process
CommandLine	Full command line including arguments (if auditing enabled)
ParentProcessName	Process that spawned the new process
SubjectUserName	User context running the process
NewProcessId	PID of the new process

Important Fields to Focus On

- CommandLine — most valuable field; shows exact attacker commands
- ParentProcessName — unusual parents (svchost→cmd, winword→powershell) = malicious
- Process running as SYSTEM when not expected = privilege escalation
- LOLBins: certutil, mshta, regsvr32, rundll32, wscript, cscript
- Encoded PowerShell (-EncodedCommand or -enc flag) = obfuscation attempt

Detection Logic: Alert on: cmd.exe or powershell.exe spawned from unusual parent (svchost, lsass, winword), net.exe /add user commands, whoami/ipconfig/arp after RDP logon, encoded PowerShell execution, LOLBin abuse patterns.

Investigation Tips

- Build parent→child process chain to understand the full kill chain
- Look for whoami, ipconfig, net user, tasklist — recon commands
- net user /add, net localgroup administrators /add = persistence
- powershell -enc or -nop -w hidden = obfuscated execution
- Check process path — malware in %TEMP%, %APPDATA%, C:\Users\Public

Common False Positives

- Legitimate admin using cmd.exe or PowerShell for maintenance
- Software installers spawning processes
- Antivirus/EDR running child processes for scanning
- Scheduled tasks creating processes

MITRE ATT&CK Mapping: T1059 — Command and Scripting Interpreter | Tactic: Execution

Sigma Rule

```
title: Suspicious Process Created by LSASS detection: selection: EventID: 4688 ParentProcessName|endswith: 'lsass.exe' NewProcessName|endswith: - 'cmd.exe' - 'powershell.exe' condition: selection level: critical
```

Splunk Query

```
index=wineventlog EventCode=4688 | search CommandLine=*net* user* /add* OR CommandLine=-enc* OR  
ParentProcessName=*svchost* NewProcessName=*cmd* | table _time, user, ParentProcessName, NewProcessName,  
CommandLine
```

Microsoft Sentinel KQL

```
SecurityEvent | where EventID == 4688 | where CommandLine has_any ("net user", "-EncodedCommand", "/add")  
or (ParentProcessName endswith "svchost.exe" and NewProcessName endswith "cmd.exe") | project  
TimeGenerated, SubjectUserName, ParentProcessName, NewProcessName, CommandLine
```

■ **SOC Note:** Enable command-line auditing via GPO or you will only see the process name, not the arguments — this halves your visibility. After enabling, 4688 becomes your most powerful post-exploitation detection event.

Interview Questions

1. What GPO setting is required to see command-line arguments in 4688?
2. Name three LOLBins commonly used in post-exploitation.
3. Why is svchost.exe spawning cmd.exe suspicious?
4. What does PowerShell -enc flag indicate?

Event ID 4698 — A Scheduled Task Was Created

Definition: Generated when a new scheduled task is created. Scheduled tasks are a primary persistence mechanism used by attackers.

Purpose: Detects persistence via scheduled tasks — attacker ensures their payload runs automatically on a recurring basis.

When Generated: When schtasks.exe, Task Scheduler GUI, or COM interface creates a new task.

Normal vs Attack Log Example

```
NORMAL: EventID: 4698 | TaskName: \Microsoft\Windows\Defrag\ScheduledDefrag User: SYSTEM | Action: %windir%\system32\defrag.exe Trigger: Weekly on Sunday at 01:00
```

```
ATTACK / SUSPICIOUS: EventID: 4698 | TaskName: \WindowsUpdate\MSUpdate User: administrator | Action: C:\Users\Public\update.exe Trigger: Every 15 minutes [PERSISTENCE – MASQUERADING AS WINDOWS UPDATE]
```

Key XML Fields

Field	Description
TaskName	Name of the scheduled task — attackers often masquerade as legit names
SubjectUserName	Who created the task
TaskContent	Full XML of the task including Action (what runs) and Trigger (when)
ClientProcessId	PID of process that created the task

Important Fields to Focus On

- TaskName masquerading as Windows/Microsoft task = suspicious
- Action path in %TEMP%, %APPDATA%, C:\Users\Public = malicious
- Short trigger intervals (every 1, 5, 15 min) = C2 keepalive
- Creator is the attacker's compromised account (administrator)

Detection Logic: Alert when: task action points to non-standard path (not System32/SysWOW64), trigger interval is < 1 hour, task name mimics Windows built-in tasks, created by non-SYSTEM account on a server.

Investigation Tips

- Decode TaskContent XML to read the full action and trigger
- Cross-reference task executable hash against VirusTotal
- Check if executable existed before this event (4663/4656 for file creation)
- Look for 4699 (task deleted) shortly after — attacker covering tracks

Common False Positives

- Software updates creating maintenance tasks (Chrome, Adobe, etc.)
- IT automation tools (SCCM, Ansible, PDQ Deploy)
- Antivirus scheduled scan tasks

MITRE ATT&CK Mapping: T1053.005 — Scheduled Task/Job: Scheduled Task | Tactic: Persistence / Privilege Escalation

Sigma Rule

```
title: Suspicious Scheduled Task Creation detection: selection: EventID: 4698 TaskContent|contains: - 'AppData' - 'Temp' - 'Public' condition: selection level: high
```

Splunk Query

```
index=wineventlog EventCode=4698 | rex field=TaskContent "(?P[^\s]+)" | where match(cmd, "(?i)(appdata|temp|public|programdata)") | table _time, user, TaskName, cmd
```

Microsoft Sentinel KQL

```
SecurityEvent | where EventID == 4698 | where TaskContent has_any ("AppData", "Temp", "Public", "ProgramData") | project TimeGenerated, SubjectUserName, TaskName, TaskContent, Computer
```

■ SOC Note: Attackers love naming tasks \Microsoft\Windows* to blend in. Always decode the TaskContent XML — the Action field reveals the payload. Correlate with 4688 to find the schtasks.exe process that created it.

Interview Questions

1. How does an attacker use scheduled tasks for persistence?
2. What TaskContent field reveals the malicious payload?
3. How can you find a scheduled task that has been deleted to cover tracks?

Event ID 4720 — A User Account Was Created

Definition: Generated when a new local or domain user account is created. Attackers create backdoor accounts to maintain persistent access.

Purpose: Detects attacker persistence through rogue account creation.

When Generated: When net user /add, New-ADUser, or ADUC creates a new account.

Normal vs Attack Log Example

NORMAL: EventID: 4720 | NewAccount: svc-backup | Creator: IT-ADMIN Time: 10:30 AM | Reason: New service account Department: Operations (verified in ITSM ticket #4421)

ATTACK / SUSPICIOUS: EventID: 4720 | NewAccount: support\$ | Creator: administrator Time: 03:45 AM | No ticket | Name mimics service account From: 185.220.101.47 active session [BACKDOOR ACCOUNT]

Key XML Fields

Field	Description
TargetUserName	Name of the newly created account
SubjectUserName	Who created the account
SubjectLogonId	Links to active logon session of the creator

Important Fields to Focus On

- Account name with \$ suffix mimics computer/service account (evasion)
- Created outside business hours = suspicious
- Created by compromised admin account after external RDP logon
- No corresponding ITSM/change ticket = unauthorized

Detection Logic: Alert when: 4720 occurs outside business hours, OR created by account not in approved admin group, OR account name follows non-standard naming convention.

Investigation Tips

- Immediately check 4728/4732 — was the new account added to admin groups?
- Correlate SubjectLogonId with the attacker's active session
- Search AD for the account and note group memberships
- Check for corresponding ITSM change request

Common False Positives

- IT provisioning new employee accounts
- Service account creation for new software deployments
- Automated identity management system (SailPoint, Okta)

MITRE ATT&CK Mapping: T1136.001 — Create Account: Local Account | Tactic: Persistence

Sigma Rule

title: User Account Created Outside Business Hours detection: selection: EventID: 4720 timeframe: outside 08:00-18:00 condition: selection level: medium

Splunk Query

```
index=wineventlog EventCode=4720 | eval hour=strftime(_time, "%H") | where hour < 8 OR hour > 18 | table _time, user, TargetUserName, src_ip
```

Microsoft Sentinel KQL

```
SecurityEvent | where EventID == 4720 | extend Hour = datetime_part('Hour', TimeGenerated) | where Hour < 8 or Hour > 18 | project TimeGenerated, SubjectUserName, TargetUserName, Computer
```

■ SOC Note: Always check 4728 and 4732 within 60 seconds of 4720. Creating a user is step one — adding them to Domain Admins is step two. Most attackers do both within seconds.

Interview Questions

1. Why do attackers append \$ to backdoor account names?
2. What events should you check immediately after seeing 4720?

Event ID 7045 — A New Service Was Installed in the System

Definition: Generated by the Service Control Manager when a new service is installed. Attackers install malicious services for persistence and privilege escalation.

Purpose: Detects service-based persistence. Services auto-start on boot and run as SYSTEM unless configured otherwise.

When Generated: When sc.exe create, New-Service PowerShell cmdlet, or any program calls CreateService() Win32 API.

Normal vs Attack Log Example

```
NORMAL: EventID: 7045 | ServiceName: CrowdStrike Falcon Sensor ServiceFile:
C:\Windows\System32\drivers\CSFalconService.sys StartType: Automatic | Account: LocalSystem

ATTACK / SUSPICIOUS: EventID: 7045 | ServiceName: WindowsHelper ServiceFile:
C:\Users\Public\Downloads\svc.exe StartType: Automatic | Account: LocalSystem [MALICIOUS SERVICE]
```

Key XML Fields

Field	Description
ServiceName	Name of the installed service
ImagePath	Executable path — most important field
ServiceType	1=Kernel driver, 16=Own process, 32=Shared process
StartType	2=Automatic, 3=Manual, 4=Disabled
AccountName	Account running the service (LocalSystem = highest privilege)

Important Fields to Focus On

- ImagePath in non-standard location (%TEMP%, User dirs, Downloads) = malicious
- AccountName LocalSystem = SYSTEM-level persistence
- ServiceName mimicking legitimate services (WindowsUpdate, Spooler) = evasion
- Kernel driver service type = can bypass many AV/EDR solutions

Detection Logic: Alert when: ImagePath not in System32/Program Files, OR ServiceName matches known malware service names, OR service created immediately after RDP logon from external IP.

Investigation Tips

- Hash the ImagePath executable and check VirusTotal immediately
- Check when the executable was created (File System timeline)
- Correlate with 4688 to find the process that ran sc.exe or New-Service
- Check if service survived reboot — if yes, persistence confirmed

Common False Positives

- Legitimate software installation (AV, EDR, monitoring agents)
- Windows Update installing new components
- IT deployment of new management tools

MITRE ATT&CK Mapping: T1543.003 — Create or Modify System Process: Windows Service | Tactic: Persistence / Privilege Escalation

Sigma Rule

```
title: New Service in Suspicious Path detection: selection: EventID: 7045 ImagePath|contains: - '\Users\'
- '\Temp\' - '\Public\' condition: selection level: high
```

Splunk Query

```
index=wineventlog EventCode=7045 | where match(ServiceFileName, "(?i)(users|temp|public|appdata)") | table
_time, ServiceName, ServiceFileName, AccountName
```

Microsoft Sentinel KQL

```
Event | where EventID == 7045 | extend ImagePath=extract("[^<]+)",1,RenderedDescription) | where  
ImagePath has_any ("Users", "Temp", "Public") | project TimeGenerated, ServiceName, ImagePath, Computer
```

■ SOC Note: Services created in user-writable directories are almost always malicious. Legitimate software installs to Program Files or System32. The ImagePath is your smoking gun — hash it and check it immediately.

Interview Questions

1. Why do attackers prefer installing malicious services vs scheduled tasks?
2. What StartType value means the service starts automatically on boot?
3. How do you quickly verify if a service executable is malicious?

Quick Reference — Windows Event ID Summary

Event ID	Name	Tactic	Key Fields	Alert Threshold
4625	Failed Logon	Credential Access	LogonType, IpAddress, Status	>10 from same IP in 2 min
4624	Successful Logon	Initial Access / Lat. Move	LogonType, IpAddress, ElevatedTokenType	10 from external IP
4672	Special Privileges	Privilege Escalation	PrivilegeList, SubjectLogonId	SeDebug/SeTcb on external logon
4688	Process Creation	Execution	CommandLine, ParentProcess, User	cmd/PS from svchost/lsass
4698	Scheduled Task Created	Persistence	TaskName, TaskContent, Action	Non-System32 action path
4720	User Account Created	Persistence	TargetUserName, SubjectUserName	Outside business hours
7045	New Service Installed	Persistence / Priv Esc	ImagePath, AccountName, StartType	Path in User/Temp/Public

■ CORRELATION CHAIN: 4625 ×N → 4624 → 4672 → 4688 (recon) → 4720+4728 → 4698 → 7045 → C2

CONFIDENTIAL — FOR TRAINING PURPOSES ONLY | Do Not Distribute